

CS-860: ARTIFICIAL **INTELLIGENCE**

- **Constraint Satisfaction Problems (CSPs) – II**
 - **Efficient Solution of CSPs**
 - **Local Search**

Text : Artificial Intelligence a Modern Approach
by Stuart Russel and Peter Norvig

INSTRUCTOR:
DR. KUNWAR FARAZ AHMED KHAN

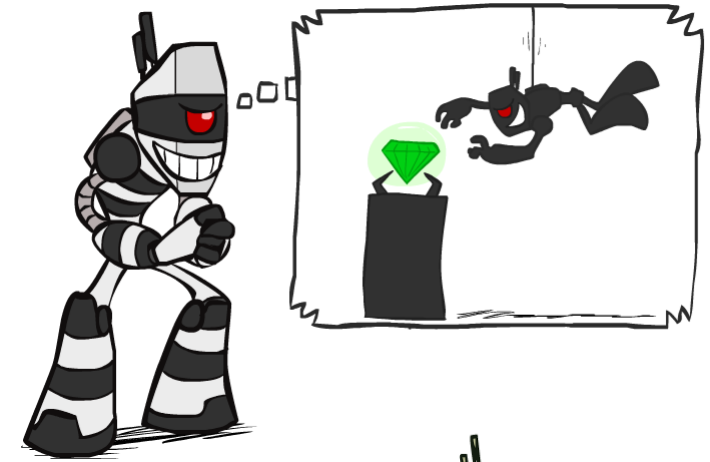
Constraint Satisfaction Problems



• Constraint Satisfaction Problems

What is search For?

- Assumptions about the world: a *single agent*, *deterministic actions*, *fully observed state*, *discrete state space*
- Planning: sequences of actions
 - The *path* to the goal is the *important* thing
 - Paths have *various costs*, depths
 - *Heuristics* give problem-specific *guidance*
- Identification: assignments to variables
 - The *goal* itself is *important*, not the path
 - All *paths* at the *same depth* (for some formulations)
 - CSPs are a *specialized class* of *identification problems*



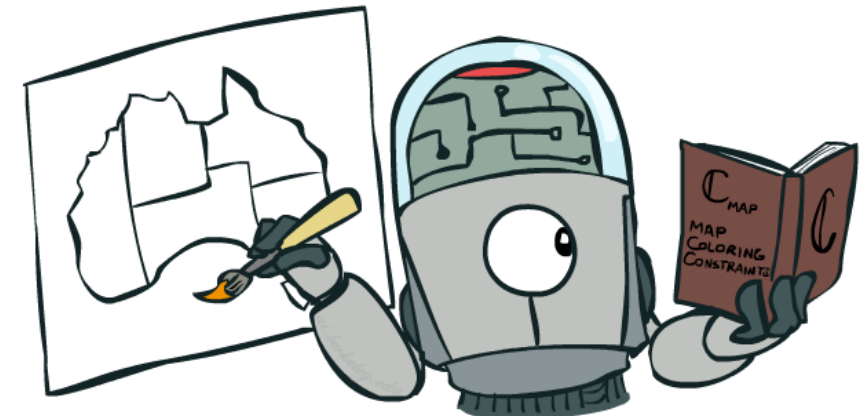
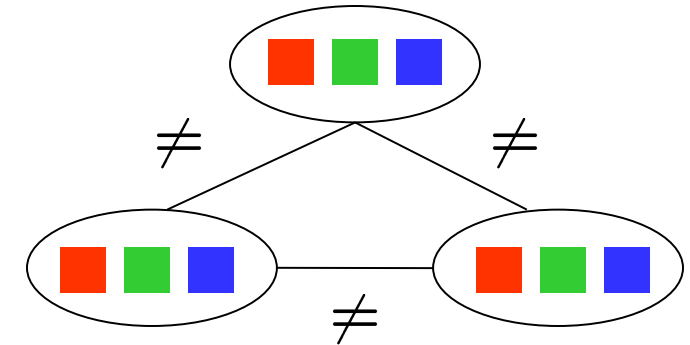
Constraint Satisfaction Problems



• CSPs Recap

CSPs Recap

- CSPs:
 - Variables
 - Domains
 - Constraints
 - Implicit (provide code to compute)
 - Explicit (provide a list of the legal tuples)
 - Unary / Binary / N-ary
- Goals:
 - Here: find any solution
 - Also: find all, find best, etc.



Constraint Satisfaction Problems



• CSPs Recap

Backtracking Search

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```



Constraint Satisfaction Problems



• CSPs Recap

Improving Backtracking

- General-purpose ideas give huge gains in speed
 - ... but it's all still NP-hard
- Filtering: Can we detect inevitable failure early?
- Ordering:
 - Which variable should be assigned next? (MRV)
 - In what order should its values be tried? (LCV)
- Structure: Can we exploit the problem structure?



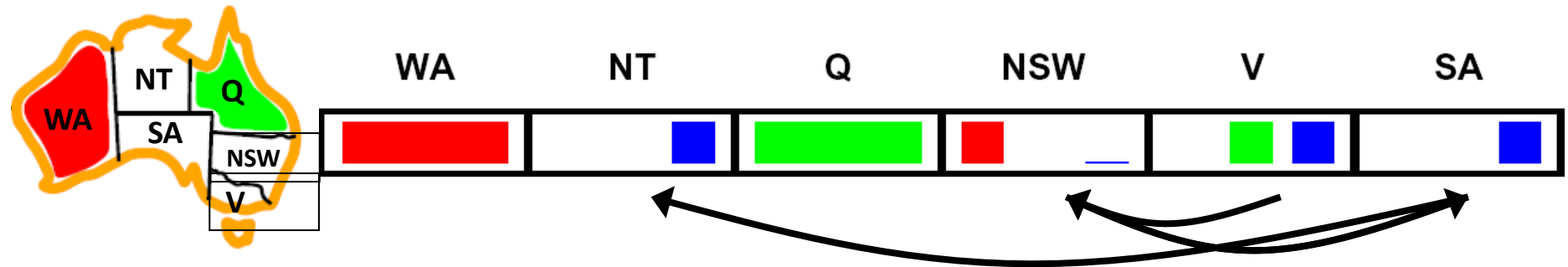
Constraint Satisfaction Problems



• CSPs Recap

Arc Consistency of an Entire CSP

- An arc $X \rightarrow Y$ is **consistent** iff for *every* x in the tail there is *some* y in the head which could be assigned without violating a constraint (head = the arrow!)
- A simple form of propagation makes sure **all** arcs are consistent:



- Arc consistency detects failure earlier than forward checking
- Important: If X loses a value, neighbors of X need to be rechecked!
- Must rerun after each assignment!

Remember:
Delete from
the tail!



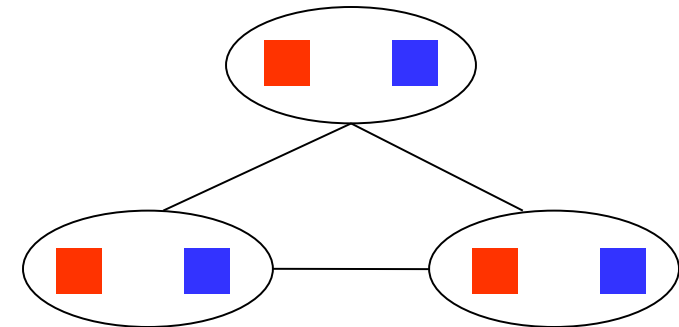
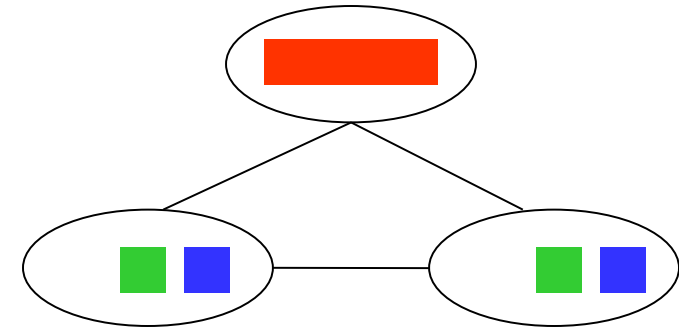
Constraint Satisfaction Problems



• CSPs Recap

Limitations of Arc Consistency

- After enforcing arc consistency:
 - Can have one solution left
 - Can have multiple solutions left
 - Can have no solutions left (and not know it)
- Arc consistency still runs inside a backtracking search!



What went wrong here?



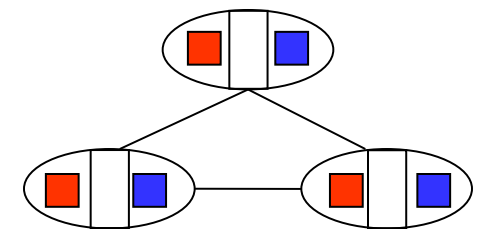
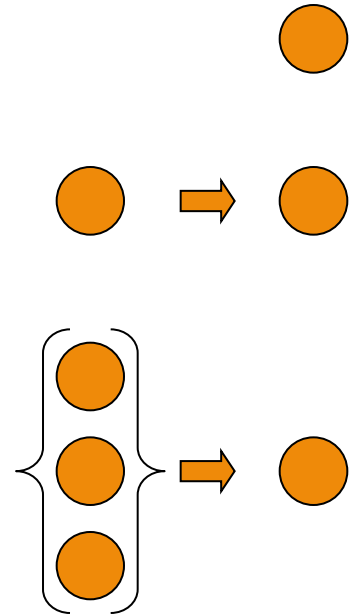
Constraint Satisfaction Problems



- CSPs Recap
- **K-Consistency**

K-Consistency

- Increasing degrees of consistency
 - 1-Consistency (Node Consistency): Each single node's domain has a value which meets that node's unary constraints
 - 2-Consistency (Arc Consistency): For each pair of nodes, any consistent assignment to one can be extended to the other
 - The way we said this before: for any assignment to the tail, there is an assignment to the head.
 - K-Consistency: For each k nodes, any consistent assignment to k-1 can be extended to the k^{th} node.
- Higher k more expensive to compute (but makes filtering better: tradeoff!)
- (You need to know the $k=2$ case: arc consistency)



Constraint Satisfaction Problems



- CSPs Recap
- **K-Consistency**
 - **Strong K-Consistency**

Strong K-Consistency

- Strong k -consistency: also $k-1$, $k-2$, ... 1 consistent
- Claim: strong n -consistency means we can solve without backtracking!
- Why?
 - Choose any assignment to any variable
 - Choose a new variable
 - By 2-consistency, there is a choice consistent with the first
 - Choose a new variable
 - By 3-consistency, there is a choice consistent with the first 2
 - ...
- Establishing strong n -consistency is great but very expensive
- Lots of middle ground between arc consistency and n -consistency! (e.g. $k=3$, called path consistency)



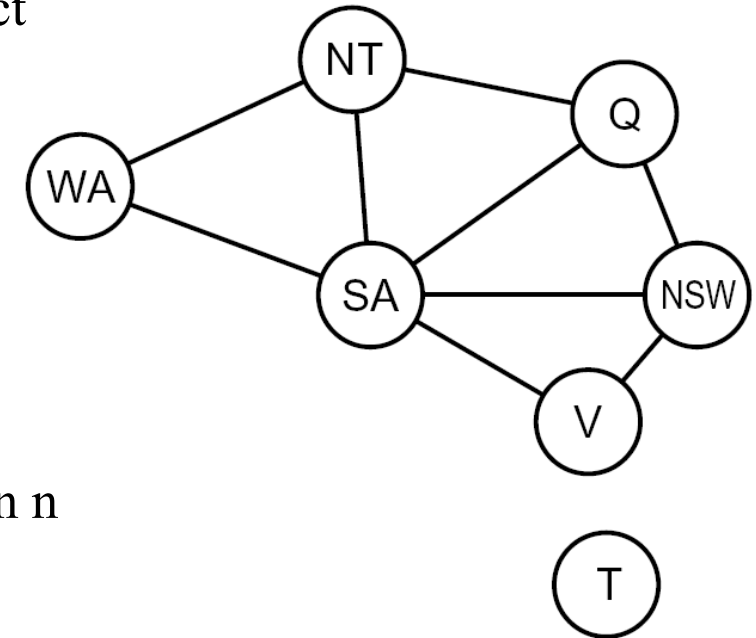
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**

Problem Structure

- Extreme case: independent subproblems
 - Example: Tasmania and mainland do not interact
- Independent subproblems are identifiable as connected components of constraint graph
- Suppose a graph of n variables can be broken into subproblems of only c variables:
 - Worst-case solution cost is $O((n/c)(d^c))$, linear in n
 - E.g., $n = 80$, $d = 2$, $c = 20$
 - $2^{80} = 4$ billion years at 10 million nodes/sec
 - $(4)(2^{20}) = 0.4$ seconds at 10 million nodes/sec
- Rarely however do graphs decompose into perfectly disconnected pieces



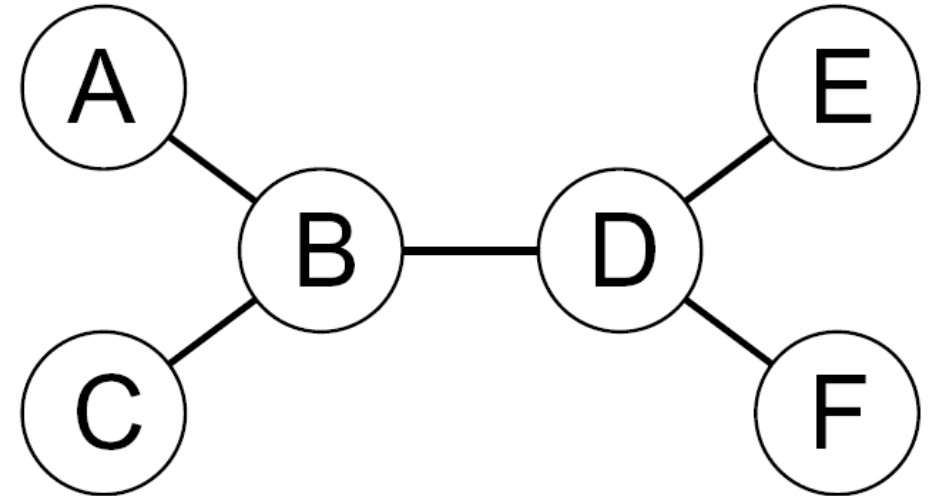
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - **Tree-Structured CSPs**

Tree-Structured CSPs

- Theorem: if the constraint graph has no loops, the CSP can be solved in $O(n d^2)$ time
 - Compare to general CSPs, where worst-case time is $O(d^n)$
- This property also applies to probabilistic reasoning (later): an example of the relation between syntactic restrictions and the complexity of reasoning



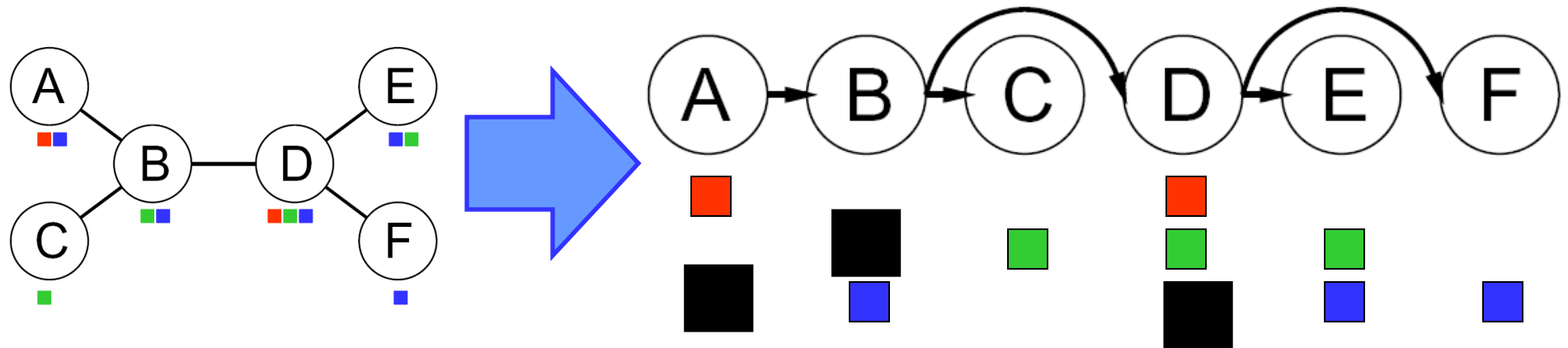
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - **Tree-Structured CSPs**

Tree-Structured CSPs

- Algorithm for tree-structured CSPs:
 - Order: Choose a root variable, order variables so that parents precede children



- Remove backward: For $i = n : 2$, apply $\text{RemoveInconsistent}(\text{Parent}(X_i), X_i)$
 - Assign forward: For $i = 1 : n$, assign X_i consistently with $\text{Parent}(X_i)$
- Runtime: $O(n d^2)$ (why?)



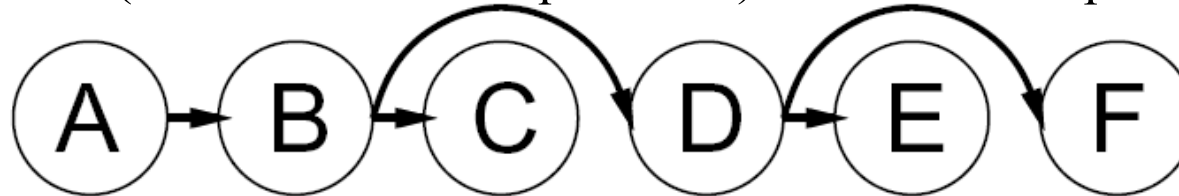
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - **Tree-Structured CSPs**

Tree-Structured CSPs

- Claim 1: After backward pass, all root-to-leaf arcs are consistent
- Proof: Each $X \rightarrow Y$ was made consistent at one point and Y 's domain could not have been reduced thereafter (because Y 's children were processed before Y)
 - Eg: once $B \rightarrow D$ is made consistent, we won't delete anything further from D 's domain (which could cause problems) because we're processing in other direction



- Claim 2: If root-to-leaf arcs are consistent, forward assignment will not backtrack
- Proof: Induction on position
 - whatever we assign to A , we know there's an assignment to B ; whatever we assign to $A+B$, we know there's an assignment to C
- Why doesn't this algorithm work with cycles in the constraint graph?
 - if 2 arcs point to same node, they are guaranteed to both have individually consistent assignments – but those assignments may conflict. (2 parents might argue!)
- Note: we'll see this basic idea again with Bayes' nets

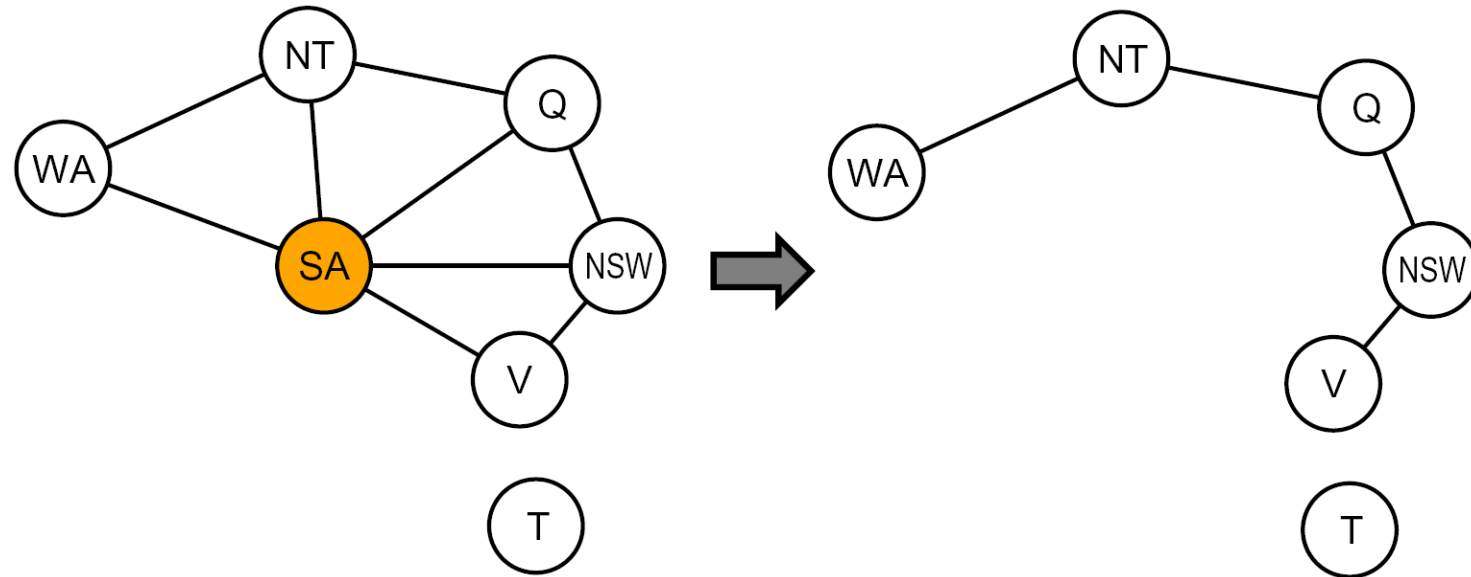


Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - **Nearly Tree-Structured CSPs**

Nearly Tree-Structured CSPs



- Conditioning: instantiate a variable, prune its neighbors' domains
- Cutset conditioning: instantiate (in all ways) a set of variables such that the remaining constraint graph is a tree
- Cutset size c gives runtime $O((d^c)(n-c)d^2)$, very fast for small c



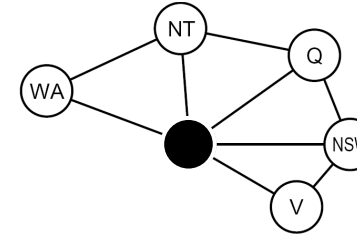
Constraint Satisfaction Problems



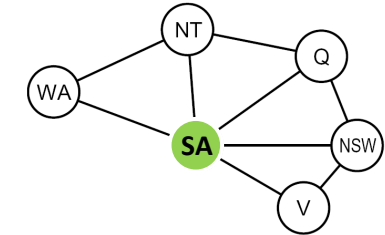
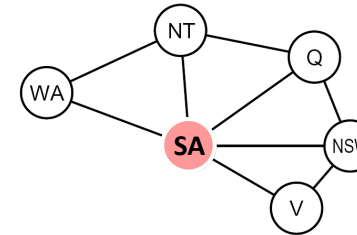
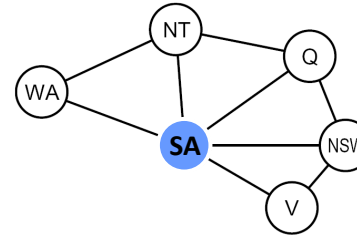
- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - **Cutset Conditioning**

Cutset Conditioning

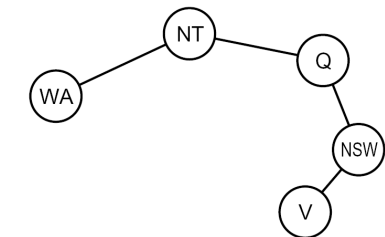
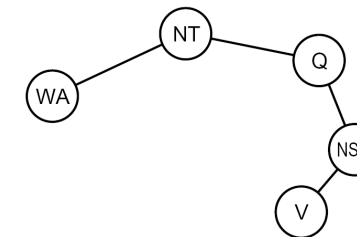
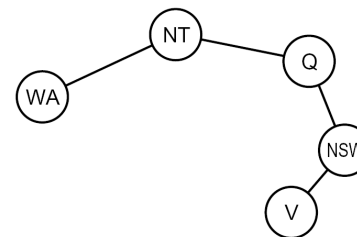
Choose a cutset



Instantiate the cutset
(all possible ways)



Compute residual CSP
for each assignment



Solve the residual
CSPs (tree structured)



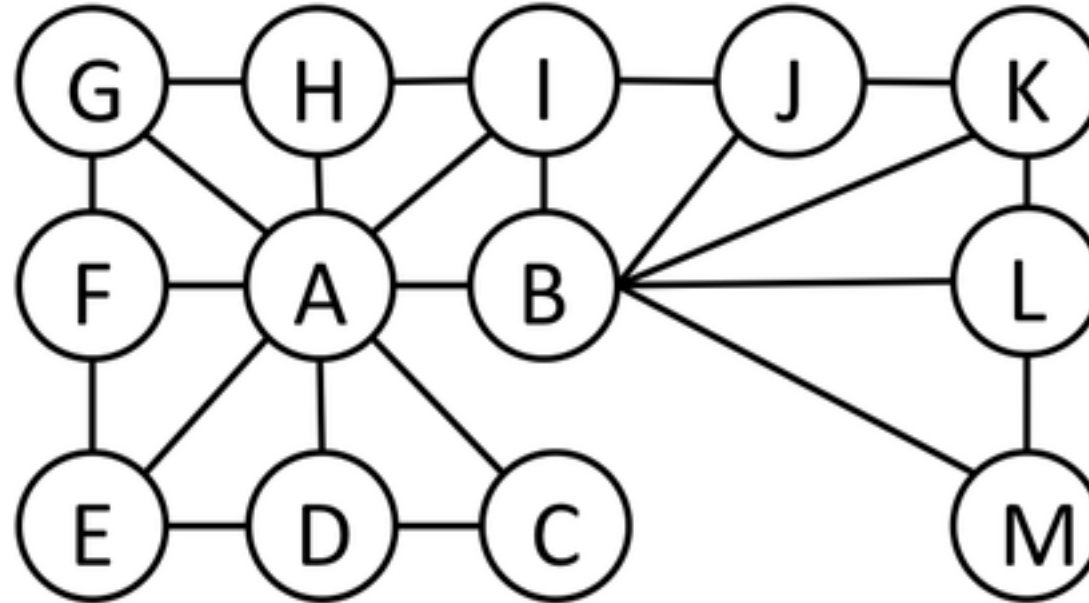
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - **Cutset Conditioning**

Cutset Conditioning

- Find the smallest cutset that gives you a tree for the graph below.



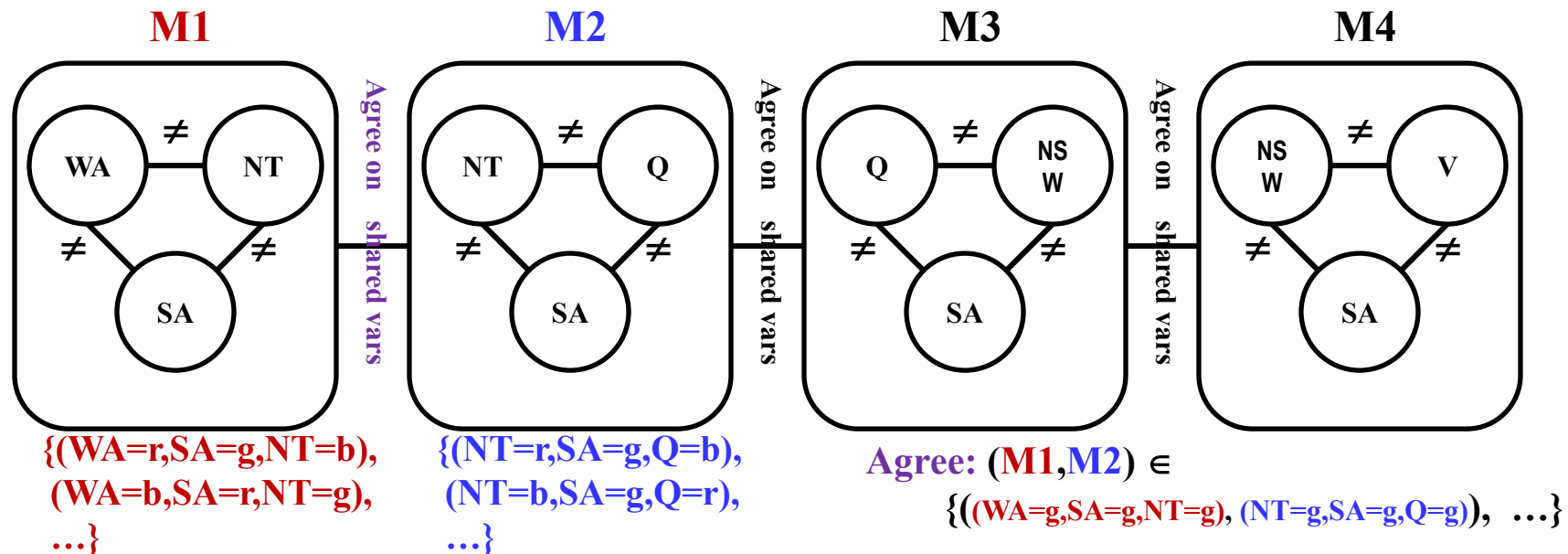
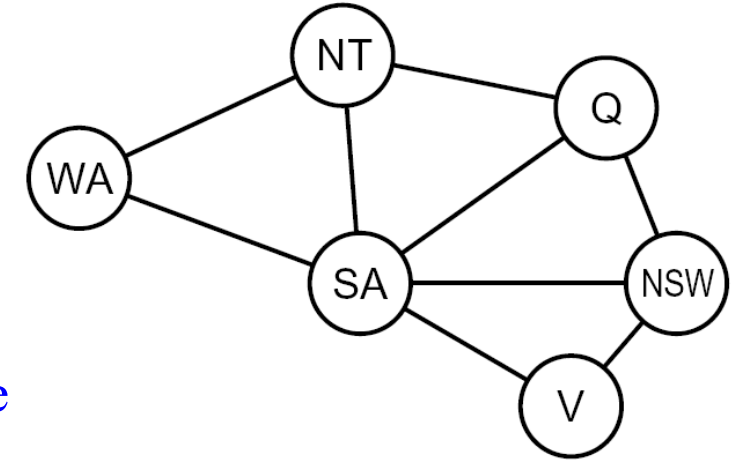
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - Cutset Conditioning
 - **Tree Decomposition**

Tree Decomposition

- Idea: create a tree-structured graph of mega-variables
- Each mega-variable encodes part of the original CSP
- Subproblems overlap to ensure consistent solutions
 - (can't just solve them separately)
- Have to set up subproblems correctly to ensure they are a tree (will not discuss how to do this)



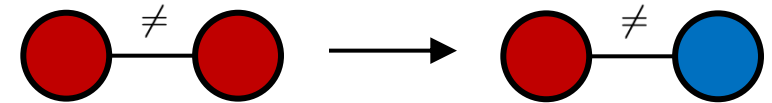
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - Cutset Conditioning
 - Tree Decomposition
 - **Iterative Algorithms for CSPs**

Iterative Algorithms for CSPs

- Local search methods typically work with *complete* states, i.e., all variables assigned
- To apply to CSPs:
 - Take an assignment with unsatisfied constraints
 - Operators *reassign* variable values
 - No fringe! Live on the edge.
- Algorithm: While not solved,
 - Variable selection: randomly select any conflicted variable
 - Value selection: min-conflicts heuristic:
 - Choose a value that violates the fewest constraints
 - i.e., hill climb with $h(n)$ = total number of violated constraints

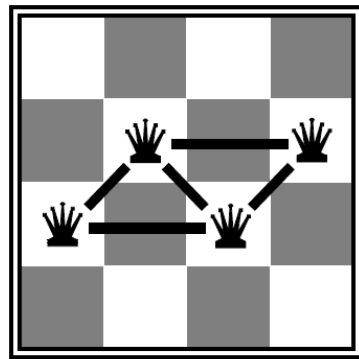


Constraint Satisfaction Problems

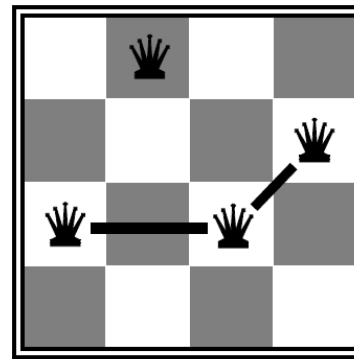
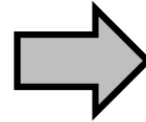


- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - Cutset Conditioning
 - Tree Decomposition
 - **Iterative Algorithms for CSPs**

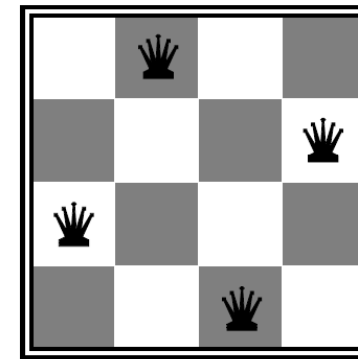
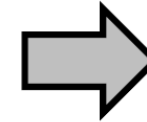
Iterative Algorithms for CSPs – Example: 4-Queens



$h = 5$



$h = 2$



$h = 0$

- States: 4 queens in 4 columns ($4^4 = 256$ states)
- Operators: move queen in column
- Goal test: no attacks
- Evaluation: $c(n)$ = number of attacks



Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - Cutset Conditioning
 - Tree Decomposition
 - **Iterative Algorithms for CSPs**

Video of Demo of Iterative Improvement – n-Queens

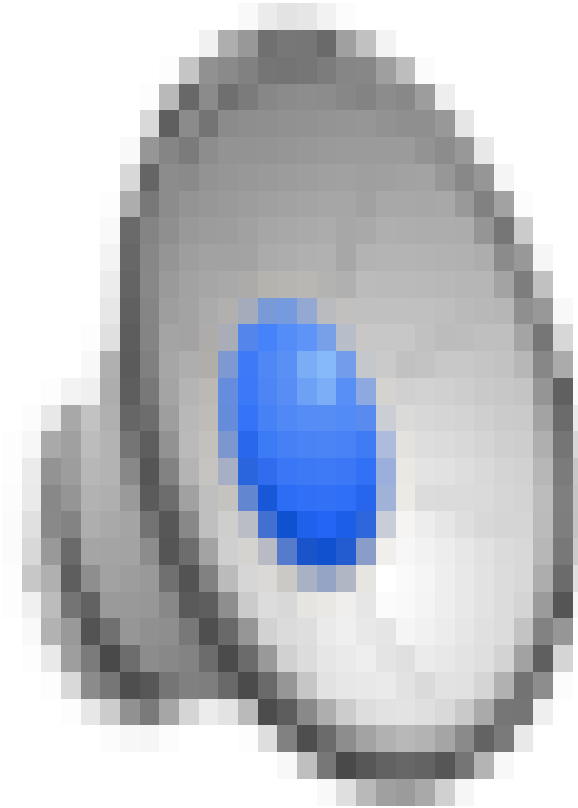


Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - Cutset Conditioning
 - Tree Decomposition
 - **Iterative Algorithms for CSPs**

Video of Demo of Iterative Improvement – Coloring



Constraint Satisfaction Problems

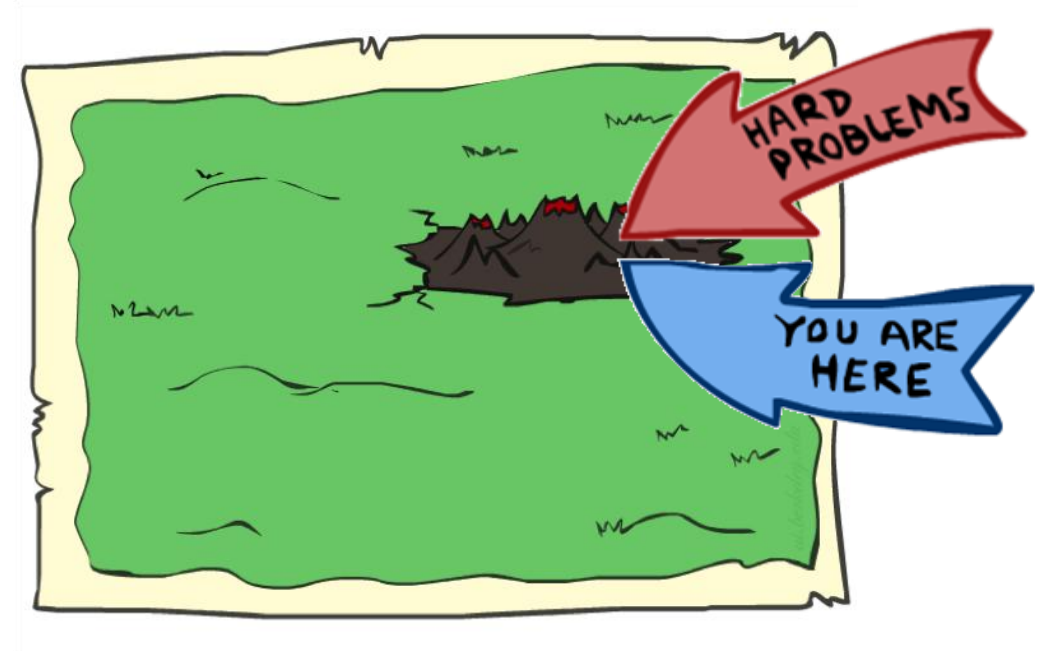
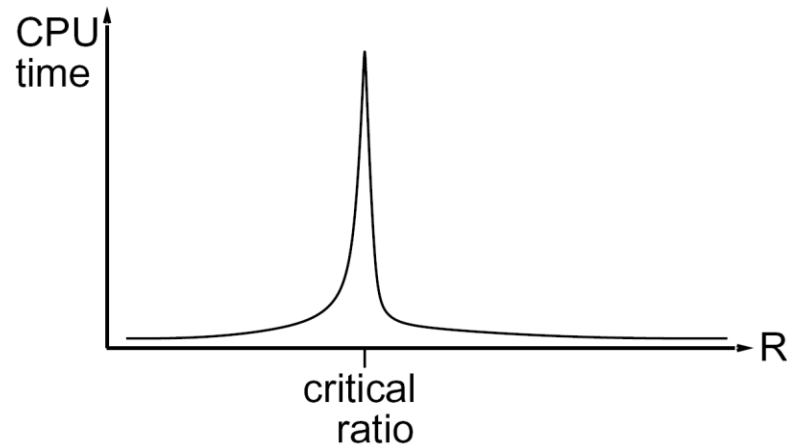


- CSPs Recap
- K-Consistency
- **Problem Structure**
 - Tree-Structured CSPs
 - Nearly Tree-Structured CSPs
 - Cutset Conditioning
 - Tree Decomposition
 - **Iterative Algorithms for CSPs**

Performance of Min-Conflicts

- Given random initial state, can solve n-queens in almost constant time for arbitrary n with high probability (e.g., $n = 10,000,000$)!
- The same appears to be true for any randomly-generated CSP *except* in a narrow range of the ratio

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$



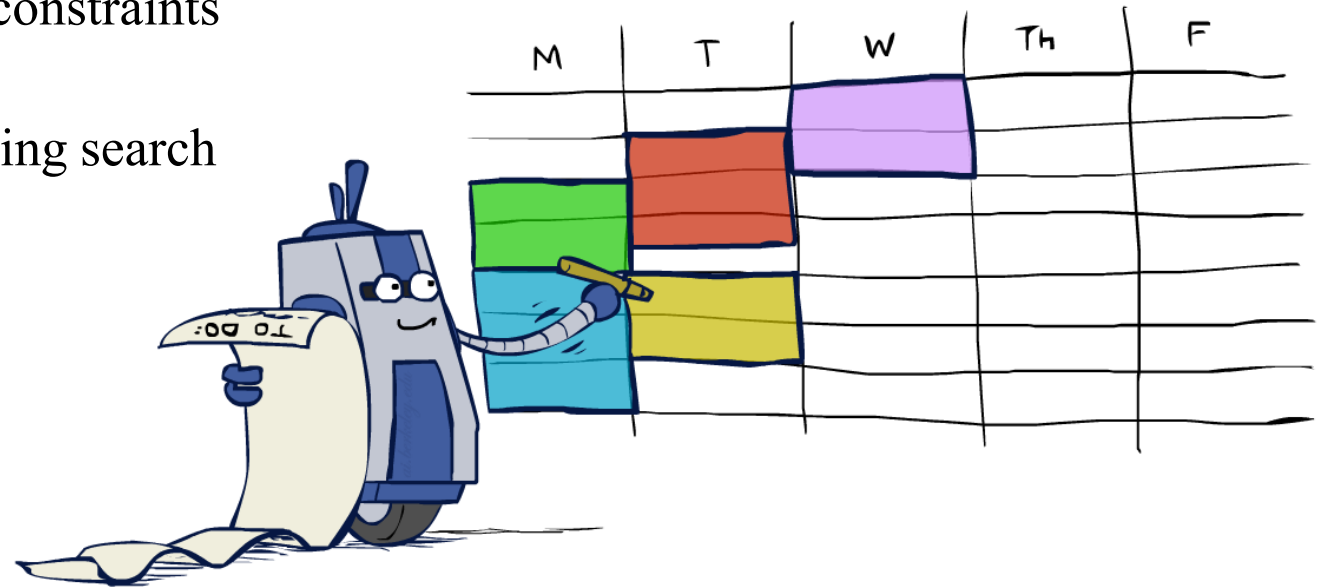
Constraint Satisfaction Problems



- CSPs Recap
- K-Consistency
- Problem Structure
- **CSP Summary**

CSP Summary

- CSPs are a special kind of search problem:
 - States are partial assignments
 - Goal test defined by constraints
- Basic solution: backtracking search
- Speed-ups:
 - Ordering
 - Filtering
 - Structure
- Iterative min-conflicts is often effective in practice



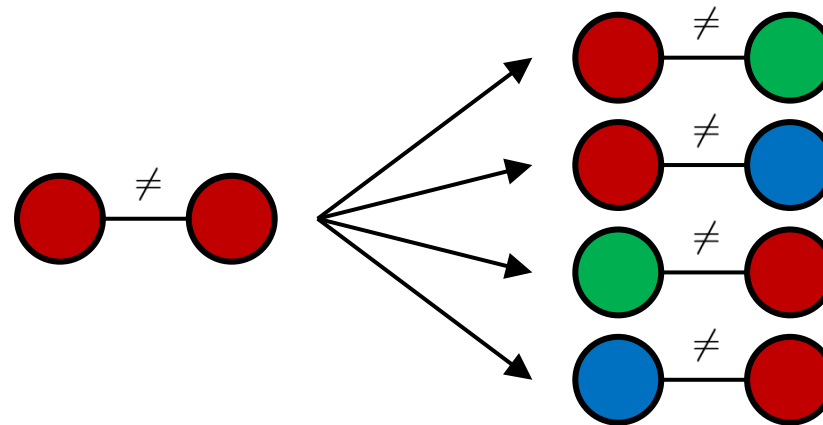


Local Search

- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**

Local Search

- Tree search keeps unexplored alternatives on the fringe (ensures completeness)
- Local search: improve a single option until you can't make it better (no fringe!)
- New successor function: local changes



- Generally, much faster and more memory efficient (but incomplete and suboptimal)
- Many machine learning algorithms use some form of local search



Local Search



- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**
 - Hill Climbing

Hill Climbing

- Simple, general idea:
 - Start wherever
 - Repeat: move to the best neighboring state
 - If no neighbors better than current, quit
- What's bad about this approach?
 - Complete?
 - Optimal?
- What's good about it?
 - Easy to apply

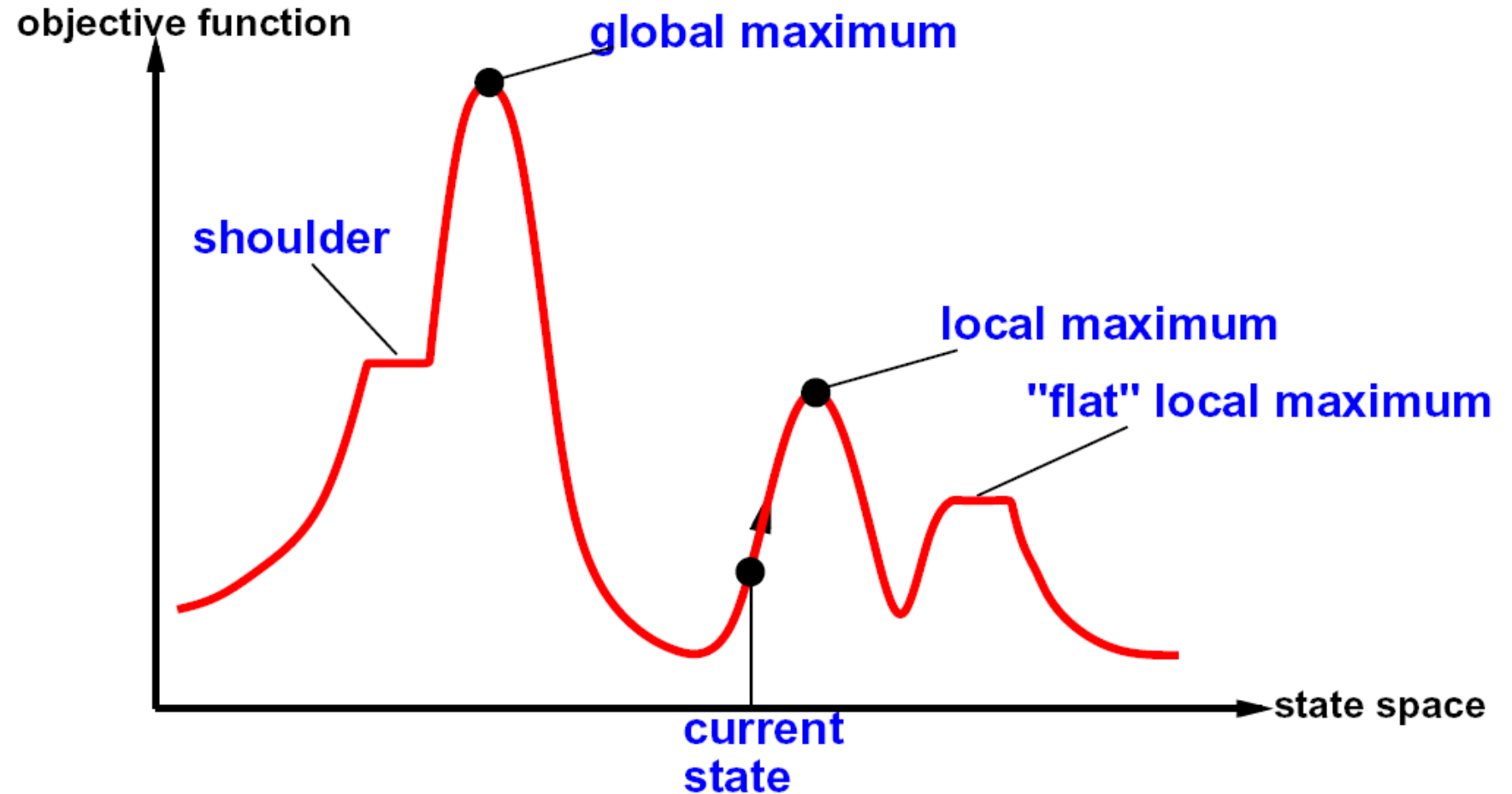


Local Search



- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**
 - Hill Climbing

Hill Climbing Diagram

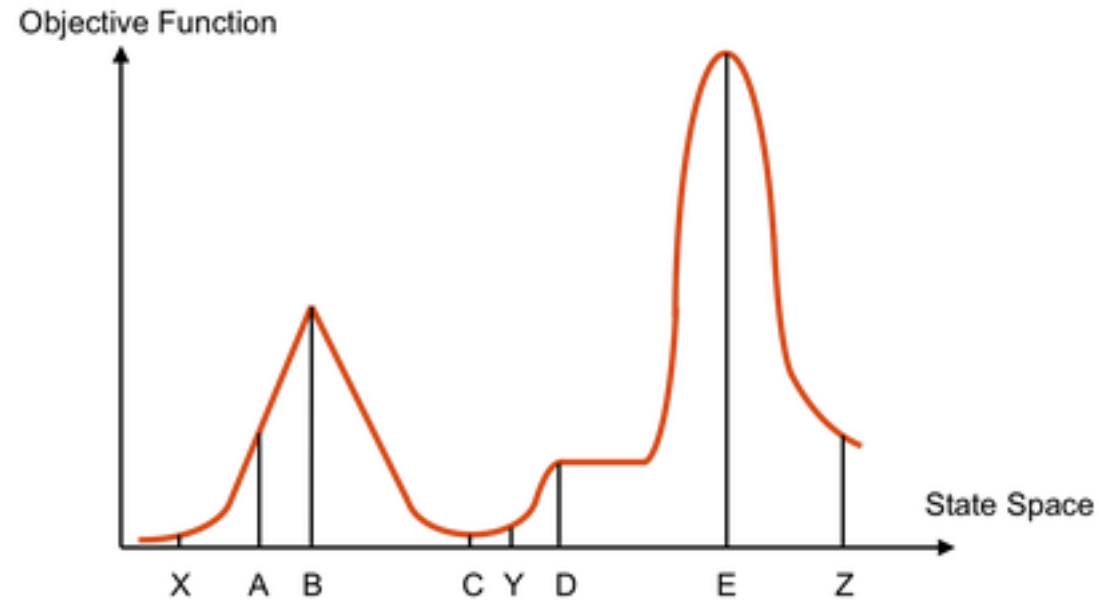


Local Search



- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**
 - Hill Climbing

Hill Climbing Diagram



Starting from X, where do you end up ?

Starting from Y, where do you end up ?

Starting from Z, where do you end up ?



Local Search



- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**
 - Hill Climbing
 - **Simulated Annealing**

Simulated Annealing

- Idea: Escape local maxima by allowing downhill moves
 - But make them rarer as time goes on

function **SIMULATED-ANNEALING**(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

local variables: *current*, a node

next, a node

T, a “temperature” controlling prob. of downward steps

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

for *t* $\leftarrow$ 1 **to** ∞ **do**

T $\leftarrow$ *schedule*[*t*]

if *T* = 0 **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow$ VALUE[*next*] – VALUE[*current*]

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$



Local Search



- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**
 - Hill Climbing
 - **Simulated Annealing**

Simulated Annealing

- Theoretical guarantee:
 - Stationary distribution:

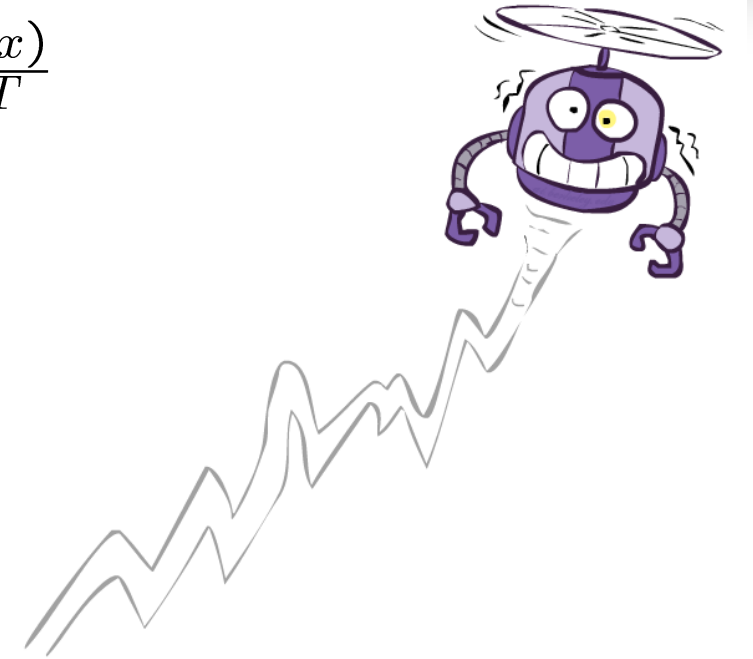
$$p(x) \propto e^{-\frac{E(x)}{kT}}$$

- If T decreased slowly enough,
 - will converge to optimal state!

- Is this an interesting guarantee?

- Sounds like magic, but reality is reality:

- The more downhill steps you need to escape a local optimum, the less likely you are to ever make them all in a row
 - People think hard about *ridge operators* which let you jump around the space in better ways

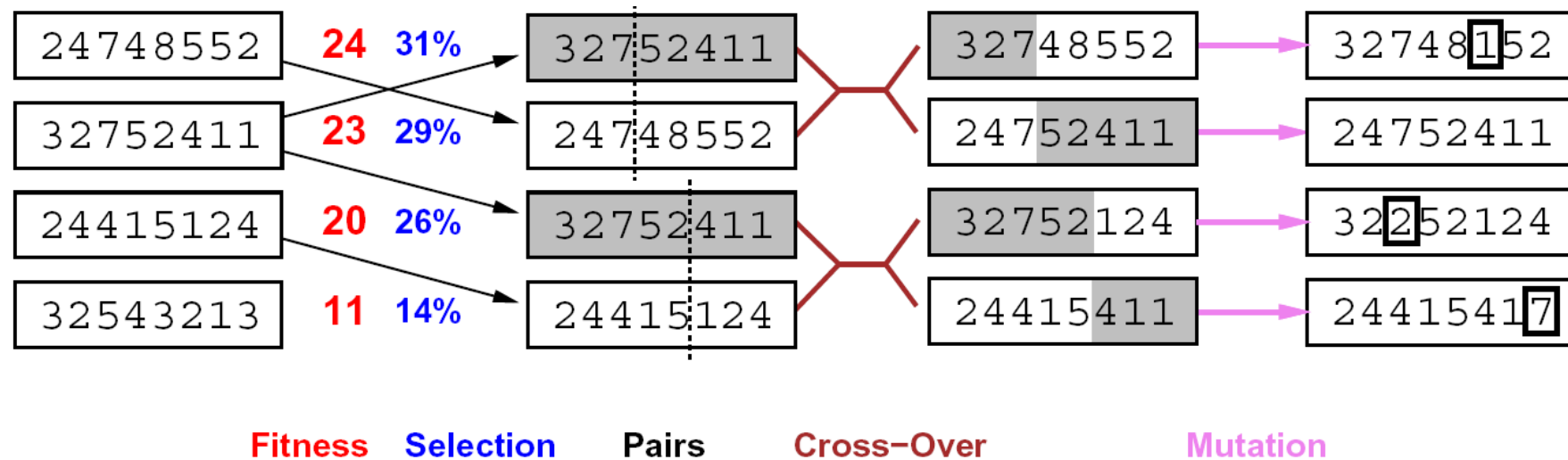




Local Search

- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**
 - Hill Climbing
 - Simulated Annealing
 - **Genetic Algorithms**

Genetic Algorithms



- Genetic algorithms use a natural selection metaphor
 - Keep best N hypotheses at each step (selection) based on a fitness function
 - Also have pairwise crossover operators, with optional mutation to give variety
- Possibly the most misunderstood, misapplied (and even maligned) technique around



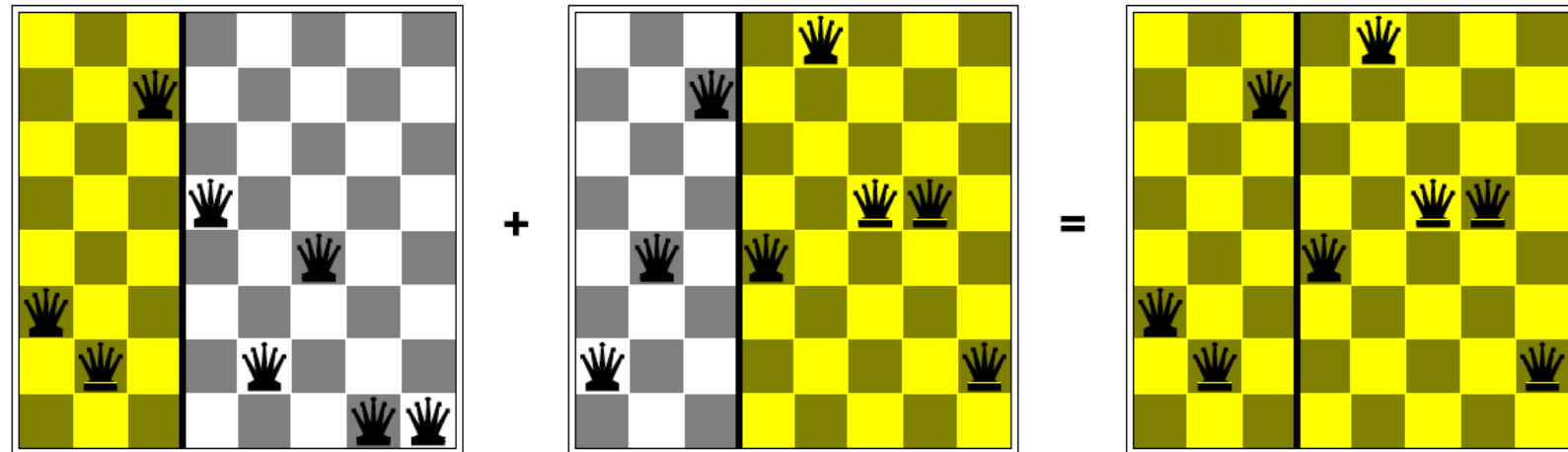
Local Search



- CSPs Recap
- K-Consistency
- Problem Structure
- CSP Summary
- **Local Search**
 - Hill Climbing
 - Simulated Annealing
 - **Genetic Algorithms**

Genetic Algorithms

- **Example: N-Queens**



- Why does crossover make sense here?
- When wouldn't it make sense?
- What would mutation be?
- What would a good fitness function be?

